Inteligencia Artificial

Camila Richter 23183

Marinés García 23391

Laboratorio 8

Task 1

Responda las siguientes preguntas de forma clara, concisa y argumentada. No basta con definir: se espera que usted conecte el concepto con sus implicaciones prácticas o estratégicas.

1. En clase discutimos dos heurísticas vitales: MCV (Variable Más Restringida) y LCV (Valor Menos Restringido). Explique con sus propias palabras por qué la heurística MCV busca que el algoritmo "falle lo más rápido posible", mientras que la heurística LCV busca que el algoritmo "tenga éxito lo más rápido posible". ¿Por qué esta combinación es tan poderosa en un árbol de Backtracking?

La heurística MCV busca que el algoritmo falle lo más rápido posible porque existe más poda al asignar cada varaible que se tiene y esta es la más acorralada por sus vecinos en el Factor Graph. En cambio, el LCV solo necesita elegir algún valor que tenga más probabilidad de conducir a una solución o cuyo arreglo de opciones posibles sea el más pequeño. Así pues, esta combinación es tan poderosa en un árbol de Backtracking porque MCV reduce las ramas 'inútiles' y LCV facilita tener soluciones rápidas; juntas hacen que el algoritmo sea más eficiente pues se exploran menos ramas y se hace menos Backtracking.

2. Suponga que está diseñando un CSP para asignar turnos a médicos en un hospital. Tiene una restricción que dice: "El Dr. Pérez y la Dra. Gómez no pueden trabajar en el mismo turno, a menos que sea fin de semana".

a. ¿Cómo modelaría esto utilizando la notación de Factores vista en clase (ej. $f_j(X) \in \{0,1\}$)? Defina cuáles serían las variables, sus dominios y escriba lógicamente la condición que hace que el factor devuelva 0 o 1.

Variables: Dr. Pérez (P), Dra. Gómez (G) y Día de la semana (D)

trabaja (T) y no trabaja (N)

Dominios: D ∈ {Lunes, Martes, Miércoles, Jueves, Viernes, Sábado, Domingo}

Factores de Condición: El Dr. Pérez y la Dra. Gómez no pueden trabajar en el mismo turno, a menos que sea fin de semana (sábado o domingo)

$$f(P,\ G,\ D) = \{1\ si\ (P \neq G) \\ \vee\ (D = lunes,\ martes,\ miercoles,\ jueves,\ viernes);\ \}$$

$$f(P,\ G,\ D) = \{0\ si\ (P = G) \wedge (D = sabado,\ domigo)\ \}$$

3. Contraste Backtracking Search con Local Search (ICM). Si usted está diseñando el sistema de emparejamiento (matchmaking) para un videojuego con millones de usuarios en línea, ¿cuál de los dos algoritmos elegirían y por qué estaría dispuesto a sacrificar la "optimalidad exacta"?

Para un sistema de *matchmaking* con millones de usuarios en línea, elegiría Local Search en lugar de Backtracking Search, ya que este último, aunque asegura que se va a encontrar la solución óptima, tiene un costo computacional exponencial que lo en una mala solución a gran escala. En cambio, Local Search permite encontrar soluciones suficientemente buenas en tiempos reducidos, esto es importante en un entorno dinámico donde los jugadores entran y salen constantemente y se necesitan respuestas en tiempo real.

Task 2 – Agente Connect Four con TD Learning

Usted es el Arquitecto Cloud de una empresa. Debe desplegar 8 Microservicios ($M1$ a $M8$) en un clúster que cuenta con 3 Servidores Físicos ($S1$, $S2$, $S3$). Debe encontrar una asignación de cada microservicio a un servidor que respete estrictamente las siguientes restricciones:

1. Capacidad (Restricción Global): Ningún servidor puede alojar más de 3 microservicios simultáneamente. Si se asignan 4 a un servidor, el peso de esa asignación es 0.

2. Anti-Afinidad (Restricciones Binarias): Por motivos de redundancia y seguridad, las siguientes parejas de microservicios no pueden estar en el mismo servidor:

a. ($M1$, $M2$) - Ej: Base de datos primaria y su réplica.

b. ($M3$, $M4$)

c. ($M5$, $M6$)

d. ($M1$, $M5$)

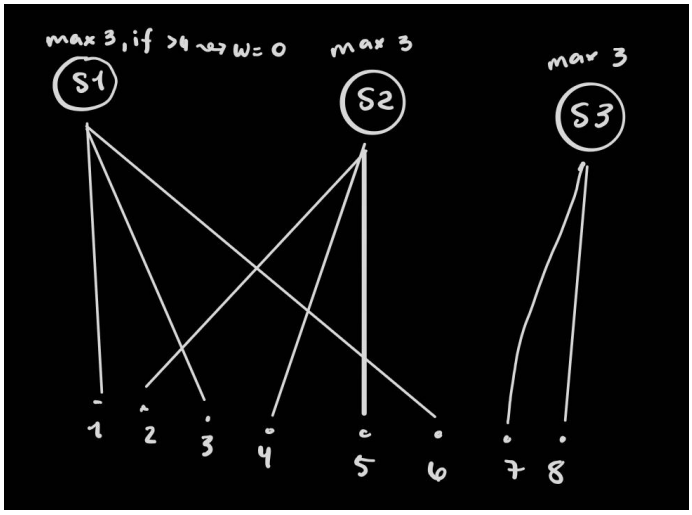
Nota: Tienen 8 variables ($M1 \ldots M8$), y el dominio de cada variable es $\{S1, S2, S3\}$

Repositorio GitHub: https://github.com/NESHGP04/Lab8-IA

Task 2.1 - Implementación de Backtracking Search

• Implemente el algoritmo de Backtracking Search para encontrar una asignación válida (Weight = 1).

• Su algoritmo debe incluir, como mínimo, la técnica de Lookahead (Forward Checking) vista en la diapositiva 24 para detener ramas de búsqueda inválidas de forma temprana.




```
¡Solución Encontrada! Asignación válida (Weight = 1):
  M1 -> S1
  M2 -> S2
  M3 -> S1
  M4 -> S2
  M5 -> S2
  M6 -> S1
  M7 -> S3
  M8 -> S3

Verificando restricciones estricta e internamente...
1. Capacidad (máx 3): S1=3, S2=3, S3=2 -> OK
2. Anti-Afinidad M1 y M2 separados: M1 en S1, M2 en S2 -> OK
2. Anti-Afinidad M3 y M4 separados: M3 en S1, M4 en S2 -> OK
2. Anti-Afinidad M5 y M6 separados: M5 en S2, M6 en S1 -> OK
2. Anti-Afinidad M1 y M5 separados: M1 en S1, M5 en S2 -> OK
Todas las restricciones de Anti-Afinidad OK.
```

Task 2.2 - Implementación de Beam Search

Una vez entrenado el agente, haga que compita en las siguientes tres condiciones, con un mínimo de 50 partidas por

• Implemente el algoritmo de Beam Search (Diapositivas 7-9).

• Debe permitir configurar el parámetro K (Tamaño del Beam).

• Su función de "peso" o heurística para seleccionar a los mejores K candidatos en cada paso (Prune) debe priorizar asignaciones que violen la menor cantidad de restricciones posible.

Task 2.3 - Implementación de Local Search (ICM)

• Implemente el algoritmo de Búsqueda Local mediante Modos Condicionales Iterados (ICM) (Diapositiva 15).

• Inicie con una asignación completamente aleatoria (que probablemente viole muchas reglas) y modifique iterativamente un microservicio a la vez, moviéndolo al servidor que maximice el cumplimiento de las restricciones.

• Implemente un límite de iteraciones por si el algoritmo se queda atascado en un óptimo local.

```
Camila@MacBook-Pro-90 Lab8-IA % /usr/local/bin/python3 "/Users/Camila/Desktop/CAMILA UNIVERSIDAD/7SEMESTRE/InteligenciaArtificia
l/Lab8-IA/task2_3.py"
============================================================
LOCAL SEARCH - ICM
============================================================

Intento 1:
✔ Solución válida encontrada

Asignación encontrada:
  M1 -> S2
  M2 -> S1
  M3 -> S3
  M4 -> S2
  M5 -> S3
  M6 -> S1
  M7 -> S3
  M8 -> S2

Verificación:
Violaciones: 0
Camila@MacBook-Pro-90 Lab8-IA % /usr/local/bin/python3 "/Users/Camila/Desktop/CAMILA UNIVERSIDAD/7SEMESTRE/InteligenciaArtificia
l/Lab8-IA/task2_3.py"
============================================================
LOCAL SEARCH - ICM
============================================================

Intento 1:
✔ Solución válida encontrada

Asignación encontrada:
  M1 -> S3
  M2 -> S2
  M3 -> S2
  M4 -> S1
  M5 -> S1
  M6 -> S3
  M7 -> S2
  M8 -> S3

Verificación:
Violaciones: 0
```

Task 2.4 - Benchmarking y Conclusiones

Cree una celda final donde ejecute los 3 algoritmos para el mismo problema. Debe imprimir y comparar:

```
Camila@MacBook-Pro-90 Lab8-IA % /usr/local/bin/python3 "/Users/Camila/Desktop/CAMILA UNIVERSIDAD/7SEMESTRE/InteligenciaArtificia
l/Lab8-IA/task2_4.py"

============================================================
BENCHMARK DE ALGORITMOS
============================================================

============================================================
  Beam Search  |  K = 3
============================================================

Resultados:

Backtracking: ✔ | Tiempo: 0.00010s
Beam Search:  ✔ | Tiempo: 0.00016s
ICM:          ✔ | Tiempo: 0.00012s
Algoritmo      Estado                    Tiempo (s)
--------------------------------------------------------
Backtracking   Solución válida           0.000104
Beam Search    Solución válida           0.000163
ICM            Solución válida           0.000123
```

1. ¿Los tres encontraron una solución válida? Si Beam o Local Search fallaron (se atascaron), repórtelo.

2. Utilice la librería time para medir cuánto tardó cada uno.

3. Redacte un breve párrafo analizando los resultados empíricos observados. ¿Se cumplió lo teórico respecto a la velocidad y la exactitud?

Los resultados empíricos concuerdan con lo esperado teóricamente. El algoritmo de Backtracking encontró una solución válida garantizando exactitud, aunque con un tiempo mayor por su exploración exhaustiva del espacio de búsqueda. Beam Search también logró encontrar una solución válida de manera eficiente al limitar el número de candidatos explorados. Por otro lado, el algoritmo de Local Search fue el más rápido y también encontró una solución válida, aunque teóricamente no garantiza optimalidad y puede quedar atrapado en óptimos locales. En conclusión, Backtracking es más completo, pero más costoso, mientras que Beam Search e ICM son más rápidos y escalables, aunque sacrifican garantías de optimalidad.